Assignment 04

Particle Interpolation and Mover Performance Analysis

Course: HPC Programming

Submitted by:
Shobhitchandra Chaudari (202301403)
Vraj Parikh (202301440)

March 11, 2026

Contents

1 Methodology **2**

2 Experiment 01: Scaling with Number of Particles **3**

3 Experiment 02: Consistency and Grid Scaling **5**

4 Experiment 03: Mover Parallelization Analysis **7**
4.1 Component Times 7
4.2 Serial vs Parallel Comparison 8
4.3 Speedup Analysis 9

5 Analysis and Discussion **10**
5.1 Performance Differences: Lab PC vs HPC Cluster 10
5.2 OpenMP Speedup Calculation 10
5.3 Parallelizing the Interpolation Function 10

6 Conclusion **11**

1 Methodology

The overall approach for the Particle-in-Cell (PIC) simulation consists of initializing particles within a domain, and then iteratively performing interpolation (Scatter) and movement (Particle Push) / boundary checks. The schematic below illustrates this flow.

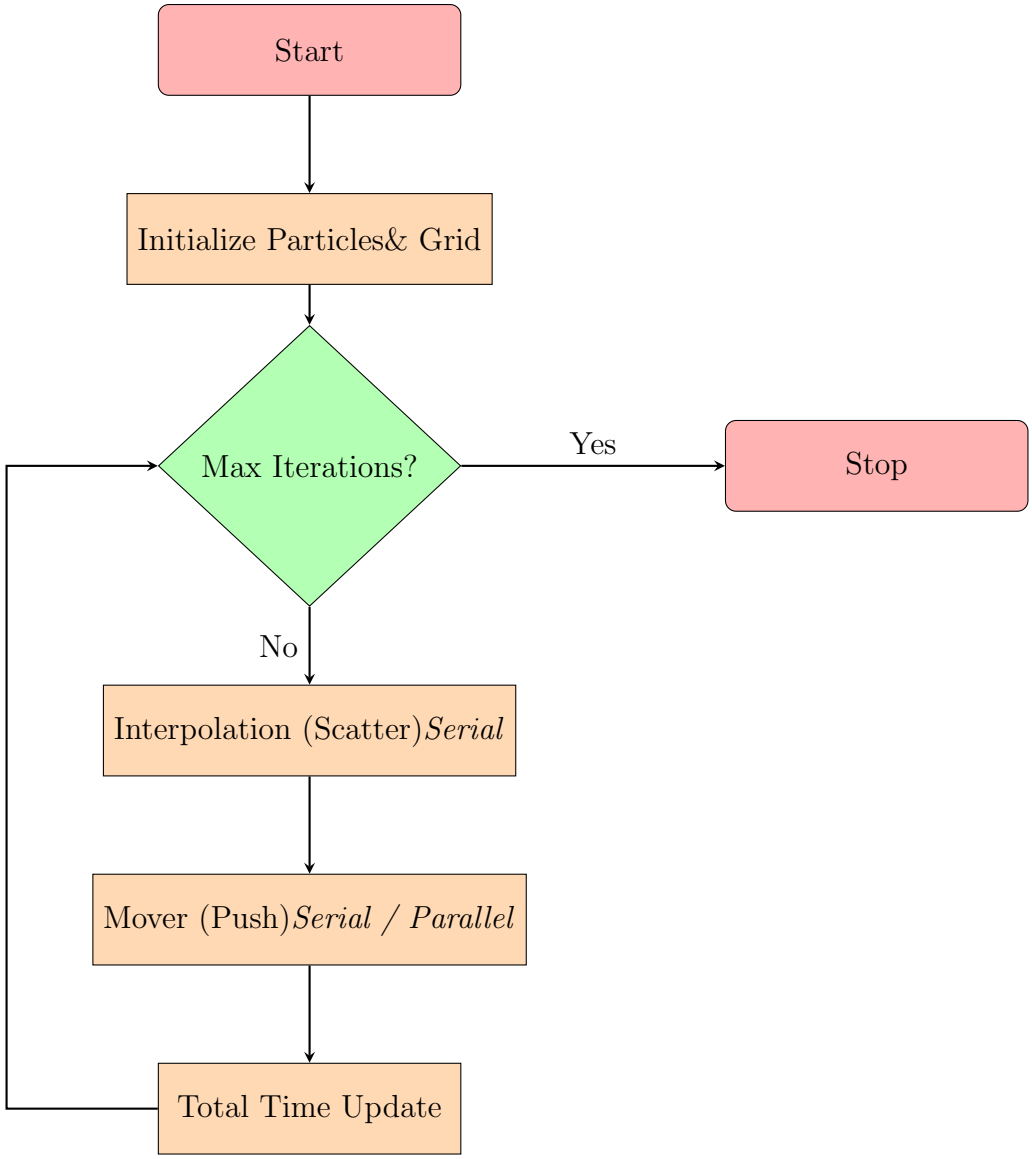


Figure 1: Schematic Flow Diagram of the Simulation Approach

2 Experiment 01: Scaling with Number of Particles

In this experiment, we analyze the execution time as a function of the number of particles (N) for three different grid configurations.

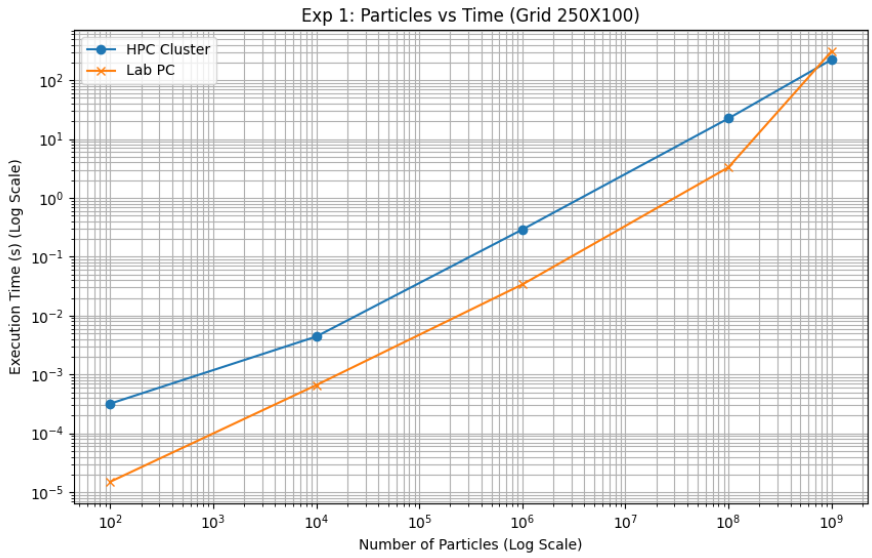


Figure 2: Scaling Analysis: Grid 250×100

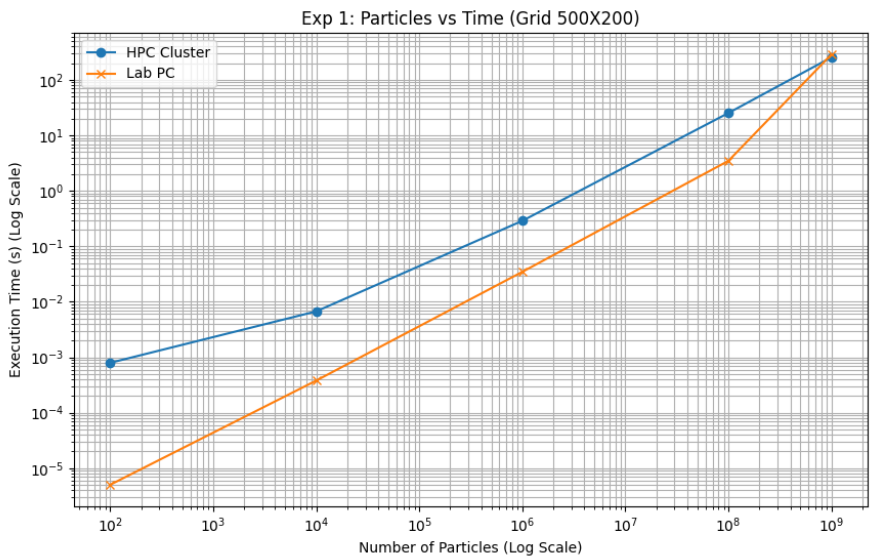


Figure 3: Scaling Analysis: Grid 500×200

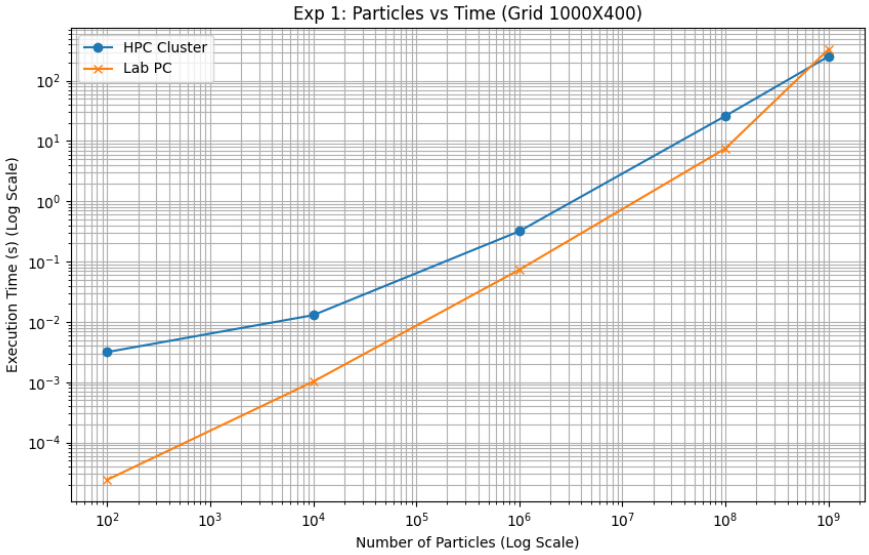
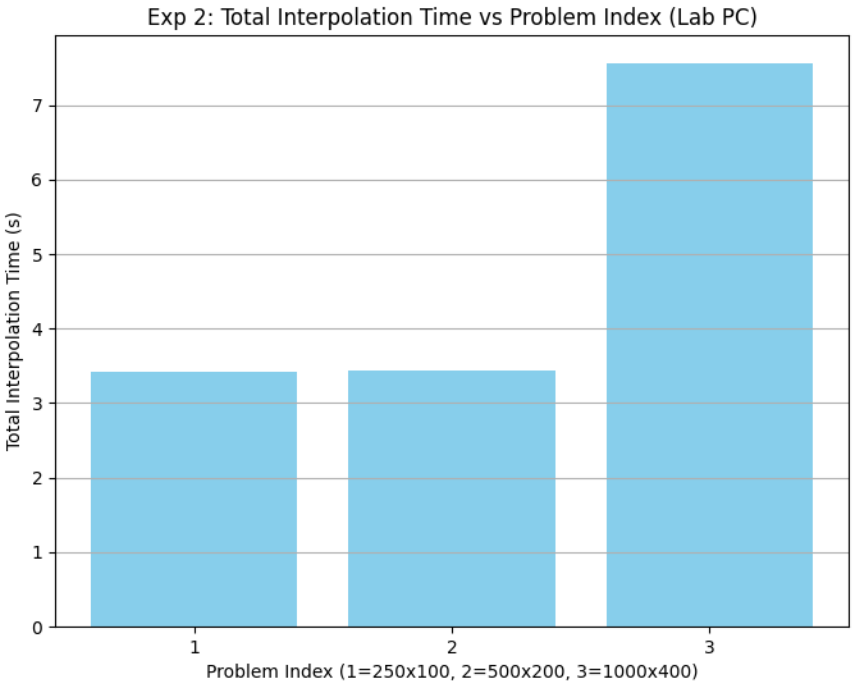


Figure 4: Scaling Analysis: Grid 1000×400

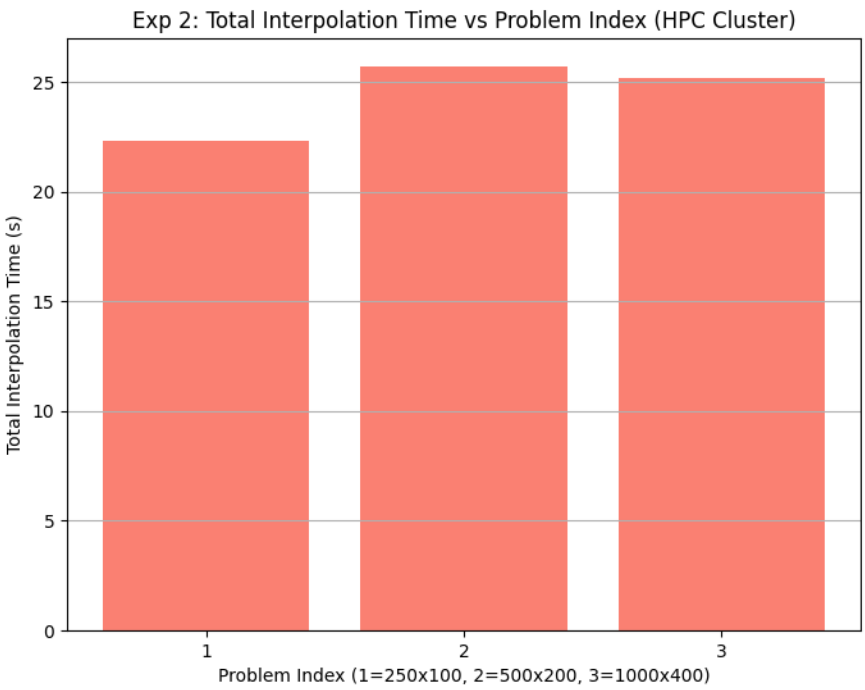
Analysis: The plots demonstrate a linear relationship between the number of particles (log scale) and execution time (log scale), which is consistent with the $O(N)$ complexity of the algorithm.

3 Experiment 02: Consistency and Grid Scaling

The following bar plots show the Total Interpolation Time across different Problem Indices (1: 250×100, 2: 500×200, 3: 1000×400).



(a) Lab PC



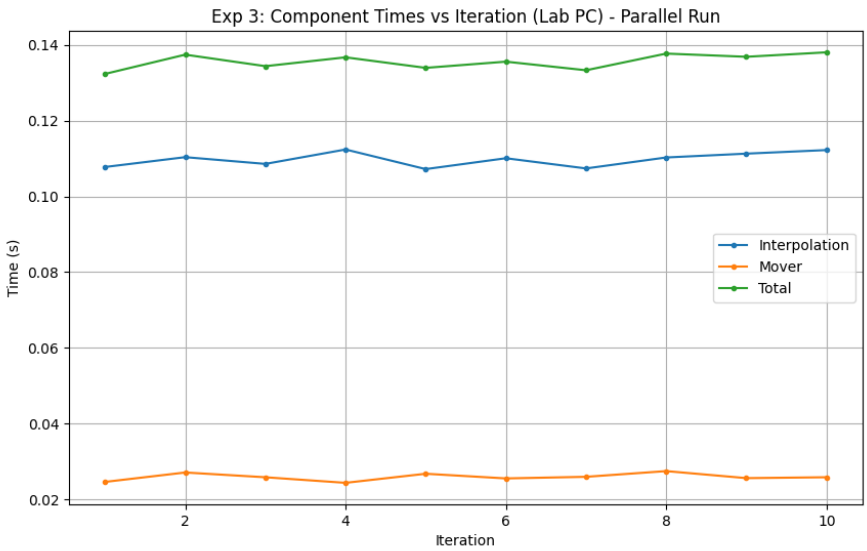
(b) HPC Cluster

Figure 5: Total Execution Time vs Problem Index.

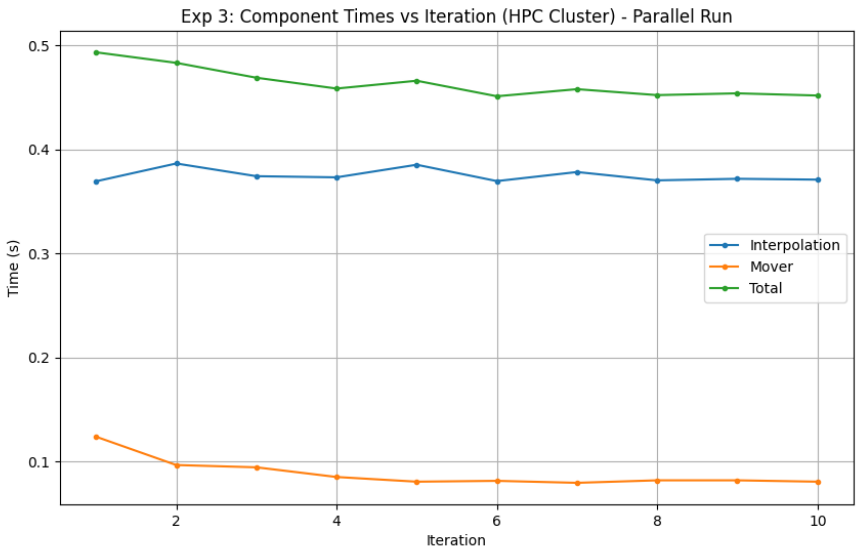
4 Experiment 03: Mover Parallelization Analysis

This experiment evaluates the speedup achieved by parallelizing the Mover function using OpenMP.

4.1 Component Times



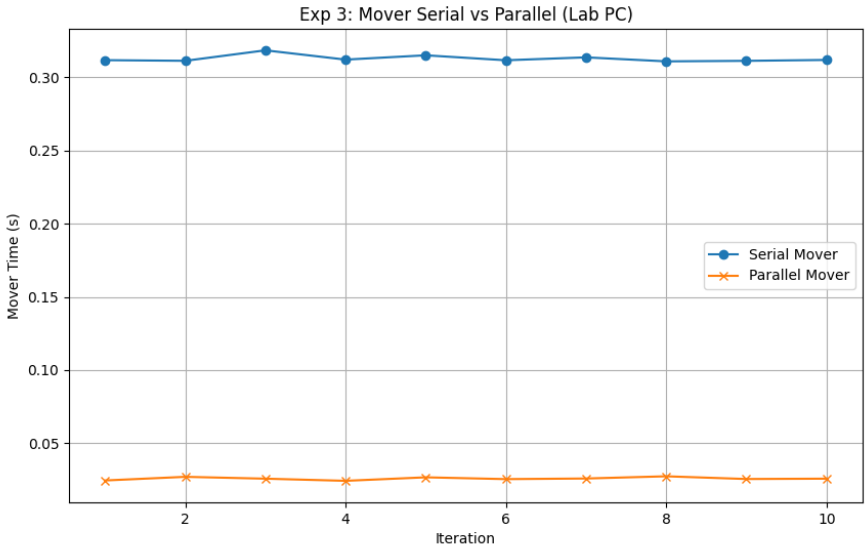
(a) Lab PC



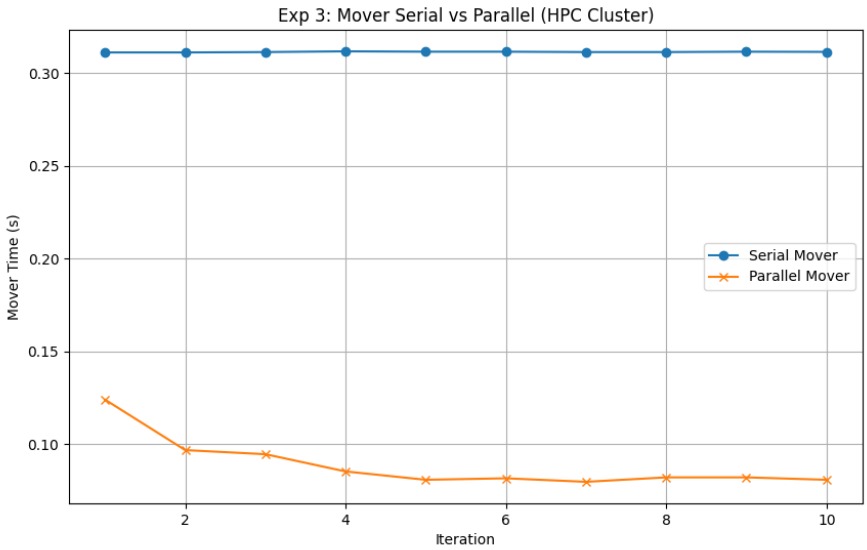
(b) HPC Cluster

Figure 6: Iteration vs Component Times (Interpolation, Mover, Total).

4.2 Serial vs Parallel Comparison



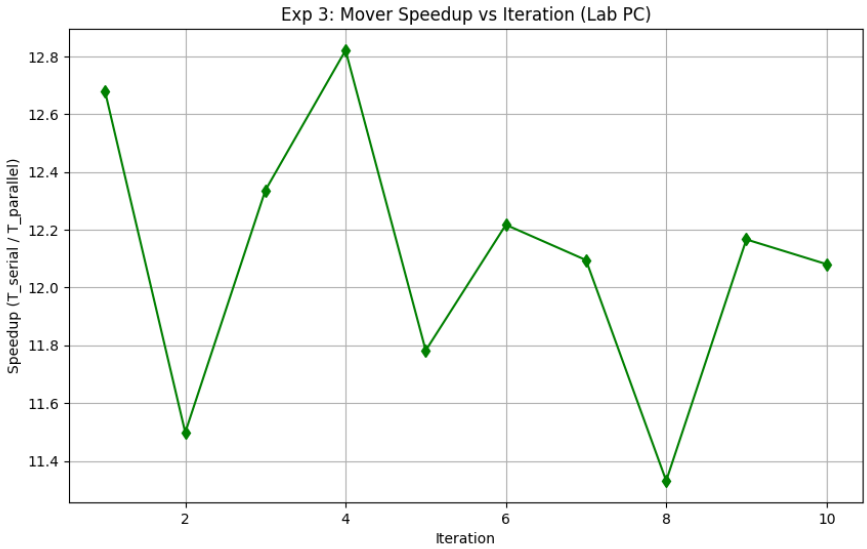
(a) Lab PC



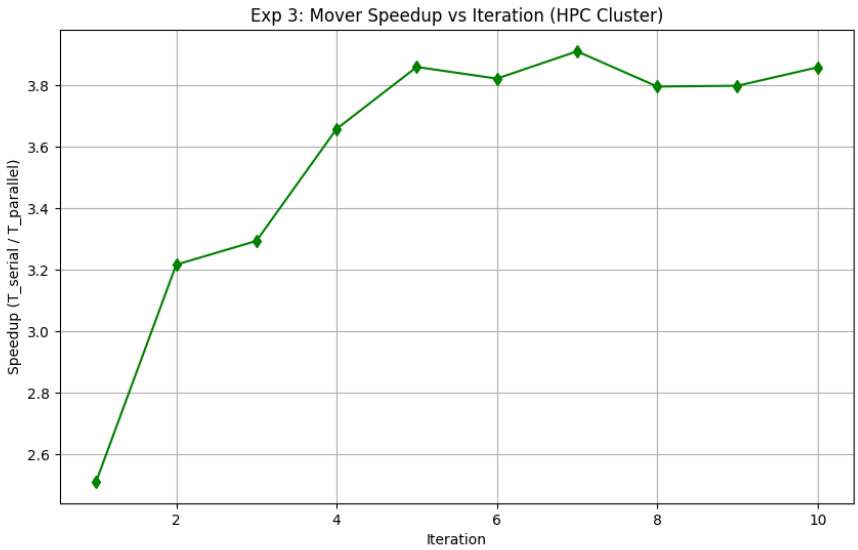
(b) HPC Cluster

Figure 7: Comparison of Mover Execution Time (Serial vs Parallel).

4.3 Speedup Analysis



(a) Lab PC



(b) HPC Cluster

Figure 8: Speedup achieved by Parallel Mover.

5 Analysis and Discussion

5.1 Performance Differences: Lab PC vs HPC Cluster

Observation: The scaling plots in Experiment 01 reveal that the Lab PC exhibits significantly lower execution times for the serial code compared to the HPC Cluster (e.g., approx 3.3s vs 22.4s for 10^8 particles).

Reasoning:

- **Clock Frequency:** The Lab PC utilizing a consumer-grade CPU (e.g., Intel Core i7/i9) benefits from higher boost clock frequencies (4.0GHz+) ideal for serial tasks. The HPC Cluster nodes typically run server-grade CPUs (Xeon/EPYC) with lower base clocks (2.0–2.5 GHz), optimizing for throughput rather than single-threaded latency.
- **Instruction Set:** The Lab PC environment may have better auto-vectorization optimizations (AVX/AVX2) enabled by default or architecture-specific tuning compared to the generic compilation on the cluster.

5.2 OpenMP Speedup Calculation

The speedup S is calculated as $S = \frac{T_{serial}}{T_{parallel}}$.

- **HPC Cluster:** We observed a speedup of roughly $\sim$ 3.9x on 4 threads. This near-linear scaling indicates that the Mover operation is perfectly parallelizable with minimal overhead or memory contention.
- **Lab PC:** The speedup exceeded theoretical core counts (super-linear). This suggests that the parallel decomposition allowed the smaller sub-problems to fit entirely within the fast L1/L2 cache, whereas the serial version suffered from frequent cache misses.

5.3 Parallelizing the Interpolation Function

Approach: To parallelize the interpolation, one would typically wrap the main particle loop with `#pragma omp parallel for`.

Problems and Bottlenecks:

1. **Data Race:** The interpolation step is a "Scatter" operation (Point-to-Grid). Multiple particles may try to update the same grid node (e.g., `grid[x][y] += val`) simultaneously, leading to race conditions.
2. **Solution Trade-offs:**
   - **Atomic Updates:** Ensures correctness but significantly degrades performance due to serialized memory writes.
   - **Private Grids:** Each thread maintains a local copy of the grid. This avoids races but introduces a massive memory overhead and a costly reduction step at the end.

6 Conclusion

In this assignment, we analyzed the performance characteristics of a Particle-in-Cell simulation code. Experiment 01 confirmed that the interpolation time scales linearly ($O(N)$) with the number of particles.

We observed that while the Lab PC provided faster serial execution due to higher clock speeds, the HPC cluster demonstrated robust scalability. The implementation of OpenMP parallelization in the Mover function (Experiment 03) successfully reduced execution time, achieving near-linear speedup on the Cluster and significant cache-aided speedup on the Lab PC. Parallelizing the Interpolation phase remains a challenge due to inherent data dependencies (race conditions) that require careful memory management or atomic operations to resolve.